Iteration: working across many columns

Tuesday, June 16, 2026

Today: Tuesday June 16

Before class

- **Perusall Reading**: *R4DS* Ch. 26 §26.1–26.3

Plan for today

- *Iteration* intro
- The copy-and-paste problem across **columns**
- `across()`: `.cols`, `.fns`, `.names`
- Selecting columns with `where()`; passing functions *without* `()`
- Anonymous functions `\(x)`, multiple functions, `if_any()` / `if_all()`
- In-class activity

Helpful links

- Syllabus · AI policy · Schedule · *R4DS* Ch. 26 · dplyr `across()` reference

Why iteration?

R's built-in iteration

Iteration is repeatedly performing the same action on different objects. In R, much of it is *implicit*. We do it without even realizing.

To double every element of a vector, you just write:

```
x <- c(1, 2, 3, 4)
2 * x
```

```
[1] 2 4 6 8
```

In many other languages you'd write a loop. In R, the operation is **vectorized**.

You've also already used tools that repeat an action over **subsets**:

- `facet_wrap()` / `facet_grid()`: one plot per group
- `group_by()` + `summarize()`: one summary per group
- `unnest_wider()` / `unnest_longer()`: one row/column per list element

Functional programming

Functional programming tools are functions that *take other functions as inputs*.

Last class, you learned to *write* functions. Now you'll *hand them to other functions* to do the repeating for you.

For the first of two iteration lectures (today), we'll focus on:

> **Doing the same thing to many columns at once.**

The key tool is dplyr's `across()`.

Modifying multiple columns

The copy-and-paste problem

Suppose we want the median of every column of this tibble:

```r
set.seed(1014)
df <- tibble(a = rnorm(10), b = rnorm(10), c = rnorm(10), d = rnorm(10))

df |> summarize(
  n = n(),
  a = median(a),
  b = median(b),
  c = median(c),
  d = median(d)
)
```

```
# A tibble: 1 x 5
      n      a      b       c     d
  <int>  <dbl>  <dbl>   <dbl> <dbl>
1    10 -0.246 -0.287 -0.0567 0.144
```

This breaks the **rule of three** from last class. With four columns, it isn't that bad, but imagine copying and pasting the same line 100 times.

across()

`across()` repeats an action over a set of columns:

```
df |> summarize(
  n = n(),
  across(a:d, median)
)
```

```
# A tibble: 1 x 5
      n      a      b       c     d
  <int>  <dbl>  <dbl>   <dbl> <dbl>
1    10 -0.246 -0.287 -0.0567 0.144
```

`across()` main arguments:

Argument	Role
`.cols`	**which** columns to iterate over (uses `select()` syntax)
`.fns`	**what** to do to each column
`.names`	(optional) **how** to name the output columns

.cols: choosing columns

`.cols` accepts everything `select()` does: `a:d`, `starts_with()`, `ends_with()`. Two selectors are especially handy with `across()`:

- **`everything()`**: every (non-grouping) column
- **`where()`**: select by *type*: `where(is.numeric)`, `where(is.character)`, `where(is.Date)`, ...

```r
set.seed(1014)
df <- tibble(
  grp = sample(2, 10, replace = TRUE),
  a = rnorm(10), b = rnorm(10), c = rnorm(10), d = rnorm(10)
)

df |>
  group_by(grp) |>
  summarize(across(everything(), median))
```

```
# A tibble: 2 x 5
    grp      a      b       c       d
  <int>  <dbl>  <dbl>   <dbl>   <dbl>
1     1 -0.244 -0.522 -0.0974 -0.251
2     2 -0.247  0.468  0.112   0.0700
```

Grouping columns (`grp`) are excluded automatically since `summarize()` already preserves them.

where(): select by type

`where()` is very helpful for handling real data with mixed column types. Here, average every **numeric** column of `flights`:

```r
flights |>
  summarize(across(where(is.numeric), \(x) mean(x, na.rm = TRUE))) |>
  glimpse()
```

```
Rows: 1
Columns: 14
$ year           <dbl> 2013
$ month          <dbl> 6.54851
$ day            <dbl> 15.71079
$ dep_time       <dbl> 1349.11
$ sched_dep_time <dbl> 1344.255
$ dep_delay      <dbl> 12.63907
$ arr_time       <dbl> 1502.055
$ sched_arr_time <dbl> 1536.38
$ arr_delay      <dbl> 6.895377
$ flight         <dbl> 1971.924
```

```
$ air_time       <dbl> 150.6865
$ distance       <dbl> 1039.913
$ hour           <dbl> 13.18025
$ minute         <dbl> 26.2301
```

You can combine selectors with Boolean logic: `!where(is.numeric)` (all non-numeric), or `starts_with("dep") & where(is.numeric)`.

.fns

We pass `median` to `across`. This is *one function handed to another function*, which is an example of functional programming.

> **Warning**
>
> Pass the **name only** with no parentheses. For example: `across(a:d, median)`, not `across(a:d, median())`.

`median()` with no argument throws an error, because you've *called* it with nothing to work on:

```
median()
```

```
Error in `median.default()`:
! argument "x" is missing, with no default
```

Adding arguments with anonymous functions

What if there are `NA`s? `median()` propagates them:

```
df_miss <- tibble(a = c(1, 2, NA, 4), b = c(NA, 5, 6, 7))
df_miss |> summarize(across(a:b, median))
```

```
# A tibble: 1 x 2
      a     b
  <dbl> <dbl>
1    NA    NA
```

We want `na.rm = TRUE` to stop this propagation. We can wrap `median()` in a tiny **anonymous function**, `\(x)`. Here, `\(x)` is shorthand for `function(x)`, aka, a function that takes argument(s) `x`:

```
df_miss |> summarize(across(a:b, \(x) median(x, na.rm = TRUE)))
```

```
# A tibble: 1 x 2
      a     b
  <dbl> <dbl>
1     2     6
```

In English, you can read this as `\(x) median(x, na.rm = TRUE)` as "given `x`, return its median, dropping NAs."

What across() expands to

It can help to picture what `across()` writes for you. This:

```
df_miss |> summarize(across(a:b, \(x) median(x, na.rm = TRUE)))
```

is exactly equivalent to:

```
df_miss |> summarize(
  a = median(a, na.rm = TRUE),
  b = median(b, na.rm = TRUE)
)
```

`across()` is just a more concise way to write the copy-pasted version. The extra bonus is that the `across()` approach prevents copy-paste mistakes.

Multiple functions at once

We can calculate the median *and* count the missings by passing a **named list** to `.fns`:

```
df_miss |>
  summarize(
    across(a:b, list(
      median = \(x) median(x, na.rm = TRUE),
      n_miss = \(x) sum(is.na(x))
```

```
    )),
    n = n()
  )
```

```
# A tibble: 1 x 5
  a_median a_n_miss b_median b_n_miss     n
     <dbl>    <int>    <dbl>    <int> <int>
1        2        1        6        1     4
```

The new columns are named `{.col}_{.fn}`, e.g. `a_median`, `a_n_miss`. This syntax is from the **glue** package, and you can adjust the naming. For example, you could switch the order and instead write `.names = "{.fn}_{.col}"`.

across() in mutate()

`across()` works well inside `mutate()` too. Recall `rescale01()` from last class. This function lets us rescale four columns in one line:

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE, finite = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}

set.seed(1014)
df <- tibble(a = rnorm(5), b = rnorm(5), c = rnorm(5), d = rnorm(5))
df |> mutate(across(a:d, rescale01))
```

```
# A tibble: 5 x 4
      a     b     c     d
  <dbl> <dbl> <dbl> <dbl>
1 0.339 1     0.291 0
2 0.880 0     0.611 0.557
3 0     0.530 1     0.752
4 0.795 0.531 0     1
5 1     0.518 0.580 0.394
```

The punchline: **write a function once**, then **apply it everywhere you want in one line** with `across()`.

Filtering with if_any() and if_all()

`across()` works in `summarize()` and `mutate()`. For `filter()`, we use similar helpers:

- **`if_any()`**: keep a row if the condition holds for *any* of the columns
- **`if_all()`**: keep a row if it holds for *all* of the columns

```
# keep only rows with NO missing values in any of these timing columns
flights |>
  filter(if_all(c(dep_time, arr_time, dep_delay, arr_delay), \(x) !is.na(x))) |>
  nrow()
```

```
[1] 327346
```

Activity

In-class activity

Open `day21-activity.R` in RStudio and work through the parts in order.

- **Part 1**: replace copy-pasted `summarize()` with `across()`; choose columns with `.cols`.
- **Part 2**: handle `NA`s with anonymous functions `\(x)`; apply *multiple* functions with a named list and control `.names`.
- **Part 3 (challenge)**: `across()` inside `mutate()` (reusing a function you write), plus `if_all()` / `if_any()` for filtering.

Activity file

Wrap-up & looking ahead

What we covered

- Iteration = the same action over many things; R gives you a lot for **free** (vectorization, faceting, `group_by`)
- **Functional programming**: hand a function to another function
- **`across(.cols, .fns, .names)`** repeats an action over columns
- `.cols` uses `select()` syntax; `everything()` and `where(is.numeric)`
- Pass the function name with **no `()`**; use `\(x) f(x, ...)` to add args
- Named list `.fns` for **multiple** functions; `{.col}_{.fn}` naming
- `if_any()` / `if_all()` bring the same idea to `filter()`

Upcoming

- **Wed Jun 17:** iterating over *files* and *list-columns* with `purrr::map()`
- **Reading before Wed:** *R4DS* §26.4–26.5
- **Homework 5**: due Friday June 19 at 11:59 p.m. on Canvas

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

Syllabus · AI policy · Schedule · *R4DS* Ch. 26 · `across()` reference